

STUDENT PROJECT ASSIGNMENT

Real-Time Motion Gesture Classifier Web App

An Edge-ML, WebAssembly, and AI-Assisted Vibe Coding Project

Assignment Objective

Students will build a single-page web application that connects to a smartphone's accelerometer sensors, feeds those live motion values into a custom Edge Impulse WebAssembly (WASM) machine learning model running locally inside the browser, and prints the predicted gestures in real-time on a responsive, futuristic visual dashboard. The finished app must be deployed to Firebase Hosting.

1. Key Learning Objectives

By completing this project, you will gain hands-on experience in the following cutting-edge tech skills:

- Train a motion-recognition machine learning model using only a smartphone sensor.
- Implement WebAssembly (WASM) models directly into client-side JS web applications.
- Integrate modern device motion browser APIs with legacy fallbacks for cross-platform support.
- Design rich web UI interfaces (vibrant dashboards, live canvas waveforms, glowing labels).
- Leverage AI coding tools (specifically Antigravity) to design, style, and code components.
- Host static web applications with backend-less logic securely using Firebase Hosting.

2. Edge Impulse Setup & Phone-Based Training

Edge Impulse allows you to collect sensor data, train neural network models, and export optimized C++ or WebAssembly libraries without needing a complex ML background. Follow these steps to train your model:

Step 1: Account & Project Setup

Go to <https://studio.edgeimpulse.com/> and sign up for a free developer account. Once logged in, create a new project named 'Motion-Gesture-Classifer'.

Step 2: Connect Smartphone

In your Edge Impulse project dashboard, go to the 'Devices' menu on the left (or click 'Connect a device' in the top right). Select 'Use your mobile phone'. A QR code will display. Scan this QR code with your mobile phone's camera. This opens a mobile data collector client in your phone's browser. Give permissions to access motion sensors when prompted.

Step 3: Collect Gesture Data

On your phone, verify the accelerometer sensor is active. Label the samples and record:

- 'Circle': Wave the phone in a wide circular pattern.
- 'Wave': Shake or wave the phone side-to-side.
- 'Idle': Keep the phone flat on a table or steady in your hand.

Collect 10 to 12 samples of 10-seconds duration for each gesture. This ensures about 2 minutes of training data per class. Ensure data is split into Training (80%) and Testing (20%).

Step 4: Design the Impulse

In the Edge Impulse Studio browser panel, go to 'Impulse design':

- Add an input block: 'Time series data' (Window size: 2000 ms, Window increase: 80 ms, Frequency: 62.5 Hz).
- Add a processing block: 'Spectral Analysis' (extracts frequencies from motion sensor readings).
- Add a learning block: 'Classification' (Keras).
- Click 'Save Impulse'.

Step 5: Generate Spectral Features

Go to 'Spectral features' in the sidebar menu. Leave default parameters, click 'Save parameters', and then click 'Generate features'. When complete, inspect the 3D Feature Explorer. You should see distinct clusters for 'Circle', 'Wave', and 'Idle'.

Step 6: Train the Classifier

Go to 'Classifier' in the sidebar. Set training epochs to 30-40 and the learning rate to 0.0005. Click 'Start training'. Ensure your validation accuracy is above 85%. If not, review labels or collect more data.

Step 7: Export WebAssembly (WASM)

Once the model has high accuracy, go to the 'Deployment' page in the sidebar. Search for 'WebAssembly' or 'WebAssembly (standalone)'. Select it and click 'Build'. A ZIP archive will download containing:

- edge-impulse-standalone.js (the JS API wrapper for the model)
- edge-impulse-standalone.wasm (the WebAssembly binary compiled with the neural network)

3. Web Application Technical Requirements

Now, you will build the web interface. Put all files inside a directory called 'public'.

Architecture

Build a single index.html file in the public directory. Place your downloaded 'edge-impulse-standalone.js' and 'edge-impulse-standalone.wasm' files in the same directory. The app should load the JS file locally using standard script tags.

Sensor Acquisition & Permission

Modern mobile browsers restrict device motion permissions. Implement a reliable sensor pipeline:

- Tier 1: W3C LinearAccelerationSensor (Standard for newer Android devices).
- Tier 2: Legacy DeviceMotionEvent (uses event.acceleration to get gravity-subtracted X, Y, Z coords).
- Tier 3: Soft Gravity Filter fallback (uses event.accelerationIncludingGravity and subtracts gravity mathematically using a high-pass software filter).

Include a prominent 'Start Sensors' button. Sensor access requires direct user interaction to trigger the permission prompt (especially on iOS).

Data Buffering & WASM Predictor

Your model was trained at a specific frequency (e.g., 62.5 Hz, which is a 16ms interval). Implement a continuous sliding buffer of accelerometer coordinates (X, Y, Z order, interleaved). Poll this buffer at the model's target frequency, flatten the data, and feed it into the Edge Impulse WASM classifier function to fetch live probabilities.

Futuristic Visual Dashboard

Develop a sleek, modern UI. The layout must feel premium and interactive. It must include:

- A live 60fps HTML5 Canvas Waveform showing real-time accelerometer readings.
- Status badges showing the current active Sensor Mode (e.g. Generic Sensor, HW Motion, or SW Filter).
- Glowing probability indicators for Circle, Wave, and Idle.
- A confidence indicator badge that lights up matching the predicted gesture class.

4. AI-Assisted Development ('Vibe Coding') Guidelines

Students are highly encouraged to build this application using an AI coding assistant, with a strong preference for Antigravity (a powerful agentic programming tool). Instead of writing code line-by-line, adopt the 'Vibe Coding' methodology:

1. Write rich prompts explaining exactly what APIs you are targeting, the filenames of your WASM module, and your specific CSS style guidelines.
2. Establish design rules for the AI: tell it to use modern fonts (e.g., Google Fonts 'Outfit' or 'Inter'), dark modes, glassmorphism (backdrop-filter: blur), vibrant gradient outlines, and clean flex/grid layouts.
3. Review and iterate: ask the AI to first build the layout, then add accelerometer APIs, then add the model inference. Run it locally and check for bugs at each iteration.

5. Firebase Hosting Setup & Deployment

Mobile browsers block access to device motion sensors unless the page is served over a secure HTTPS connection. To run the web app on your phone, you must host it. You will use Firebase Hosting for free:

1. Install Firebase CLI: Run 'npm install -g firebase-tools' in your terminal.
2. Login to your Google account: Run 'firebase login'.
3. Create a project: Go to <https://console.firebase.google.com/> and create a new project.
4. Initialize Hosting: In your local project directory, run 'firebase init hosting'.
 - Select your project from the menu.
 - Set the public directory name to 'public'.
 - Configure as single-page app: No.
5. Deploy assets: Put your index.html, JS and WASM files into the public directory, then run 'firebase deploy'.
6. Test: Open the provided HTTPS deployment URL on your phone's browser and perform the gestures.